PYTHON SUMMER PLAN • DAY 02

Operator Overloading & Properties

Make + , - , * , == , abs() , and len() work on Vector3D.

Tuesday, May 12 · 1 hour

Yuvraj Ruparel

Where we left off

You built a Vector3D class with __init__, three methods, and __repr__. It worked. Today we make it feel less like a custom class and more like a built-in type — by hooking into Python's operator syntax.

YESTERDAY

Method-style

```
v1 = Vector3D(1, 2, 3)
v2 = Vector3D(4, 5, 6)

# add componentwise — manually
v3 = Vector3D(
    v1.x + v2.x,
    v1.y + v2.y,
    v1.z + v2.z)

mag = v1.magnitude()
```

TODAY

Operator-style

```
v1 = Vector3D(1, 2, 3)
v2 = Vector3D(4, 5, 6)

# add componentwise — clean
v3 = v1 + v2

v4 = 2 * v1
same = v1 == v2
mag = abs(v1)
mag = v1.magnitude   # no parens!
```

Same math, cleaner syntax. The magic is dunder methods + @property.

Dunder methods — Python's operator hooks

Methods with double-underscore names hook into Python's syntax. Define them, and operators start working on your objects.

OPERATOR	CALLS	MEANING
`v1 + v2`	`v1.__add__(v2)`	*addition*
`v1 - v2`	`v1.__sub__(v2)`	*subtraction*
`v * 2`	`v.__mul__(2)`	*multiplication (right side)*
`2 * v`	`v.__rmul__(2)`	*reflected multiplication (left side)*
`v1 == v2`	`v1.__eq__(v2)`	*equality*
`abs(v)`	`v.__abs__()`	*absolute value / magnitude*
`len(v)`	`v.__len__()`	*length / count*
`print(v)`	`v.__repr__()`	*you already know this one*

__add__ — make + work

```python
class Vector3D:
    def __init__(self, x, y, z):
        self.x, self.y, self.z = x, y, z

    def __add__(self, other):
        return Vector3D(self.x + other.x,
                        self.y + other.y,
                        self.z + other.z)

    def __repr__(self):
        return f"Vector3D({self.x}, {self.y}, {self.z})"
```

```python
v1 = Vector3D(1, 2, 3)
v2 = Vector3D(4, 5, 6)
print(v1 + v2)
```

OUTPUT

```
Vector3D(5, 7, 9)
```

The math

$$v_1 + v_2 = (x_1+x_2,\ y_1+y_2,\ z_1+z_2)$$

Three things to notice

1. Returns a NEW Vector3D — never mutates self or other.
2. Takes one arg (self gets the left operand, other the right).
3. Python calls __add__ automatically when it sees v1 + v2.

__sub__ and __mul__ — same pattern

__sub__

```
def __sub__(self, other):
    return Vector3D(
        self.x - other.x,
        self.y - other.y,
        self.z - other.z)
```

```
v1 = Vector3D(5, 7, 9)
print(v1 - Vector3D(1, 2, 3))
```

OUTPUT

```
Vector3D(4, 5, 6)
```

__mul__ (scalar)

```
def __mul__(self, scalar):
    return Vector3D(
        self.x * scalar,
        self.y * scalar,
        self.z * scalar)
```

```
v = Vector3D(1, 2, 3)
print(v * 2)
```

OUTPUT

```
Vector3D(2, 4, 6)
```

But `2 * v` fails with TypeError. __mul__ only handles vector-on-the-left. Next slide fixes this.

__rmul__ — when the scalar comes first

*When Python sees 2 * v, it first tries int.__mul__(v), which doesn't know about Vector3D. So it then tries v.__rmul__(2). Define __rmul__ and you're covered both ways.*

```
def __rmul__(self, scalar):
    return self * scalar     # delegate to __mul__
```

```
v = Vector3D(1, 2, 3)
print(v * 2)        # __mul__
print(2 * v)        # __rmul__
```

OUTPUT

```
Vector3D(2, 4, 6)
Vector3D(2, 4, 6)
```

Why "r"?

Stands for ***reflected***. Python flips the operands and asks the right side: "Do you know how to handle this?"

Other reflected dunders:

`__radd__` `__rsub__` `__rmul__` `__rtruediv__`

__eq__ — comparing objects

Without __eq__, Python compares by identity (memory address), so v1 == v2 is always False for different objects — even with the same values.

```
def __eq__(self, other):
    return (self.x == other.x and
            self.y == other.y and
            self.z == other.z)
```

```
print(Vector3D(1, 2, 3) == Vector3D(1, 2, 3))
print(Vector3D(1, 2, 3) == Vector3D(1, 2, 4))
```

OUTPUT

```
True
False
```

Float-equality gotcha

After float math, two "equal" vectors may differ by 1e-16:

```
v = Vector3D(1,0,0)
n = v.normalize()
v == n  # might be False!
```

For physics work, prefer: abs(a - b) < 1e-9

__abs__ and __len__ — built-in functions

__abs__

Called when you write abs(v). For a vector, return the magnitude.

```
def __abs__(self):
    return self.magnitude
    # (once magnitude is a @property)
```

```
v = Vector3D(3, 4, 0)
print(abs(v))
```

OUTPUT

```
5.0
```

__len__

Called when you write len(v). For a 3D vector, return 3.

```
def __len__(self):
    return 3
    # number of components
```

```
v = Vector3D(1, 2, 3)
print(len(v))
```

OUTPUT

```
3
```

Pattern: every built-in function that needs to do something with your object asks the object to provide it via a dunder.

@property — method calls that look like attributes

magnitude doesn't take any arguments and doesn't change state — it just exposes a derived value. That's exactly what properties are for.

BEFORE

```
def magnitude(self):
    return math.sqrt(
        self.x**2 + self.y**2 + self.z**2)

v = Vector3D(1, 2, 3)
print(v.magnitude())   # parens required
```

AFTER

```
@property
def magnitude(self):
    return math.sqrt(
        self.x**2 + self.y**2 + self.z**2)

print(v.magnitude)    # no parens!
```

When to use @property

- ✓ Read-only values derived from instance state (magnitude, area, total_mass).
- ✓ Computation is cheap — callers expect attribute access to be fast.
- ✗ Don't use for expensive operations or methods that take arguments (those stay as methods).

All operators in one Vector3D

```python
import math

class Vector3D:
    def __init__(self, x, y, z):
        self.x, self.y, self.z = x, y, z

    @property
    def magnitude(self):
        return math.sqrt(self.x**2 + self.y**2 + self.z**2)

    def __add__(self, other):
        return Vector3D(self.x+other.x, self.y+other.y, self.z+other.z)
    def __sub__(self, other):
        return Vector3D(self.x-other.x, self.y-other.y, self.z-other.z)
    def __mul__(self, k):     return Vector3D(self.x*k, self.y*k, self.z*k)
    def __rmul__(self, k):    return self * k
    def __eq__(self, other):  return self.x==other.x and self.y==other.y and self.z==other.z
    def __abs__(self):        return self.magnitude
    def __len__(self):        return 3
    def __repr__(self):       return f"Vector3D({self.x}, {self.y}, {self.z})"
```

```python
a = Vector3D(1, 2, 3)
b = Vector3D(4, 5, 6)

print(a + b)
print(2 * a)
print(a == a)
print(abs(a))
print(len(a))
```

OUTPUT

```
Vector3D(5, 7, 9)
Vector3D(2, 4, 6)
True
3.7416...
3
```

TODAY'S EXERCISE · 60 MIN

Upgrade Vector3D

Take yesterday's class and add operator overloading, plus convert magnitude into a property.

FOR REFERENCE

Addition

$v_1 + v_2 = (x_1+x_2, \ldots)$

Scalar multiply

$k \cdot v = (kx, ky, kz)$

Magnitude (property)

$\|v\| = \sqrt{(x^2 + y^2 + z^2)}$

Every dunder must return a NEW Vector3D — never mutate self.

Requirements

1	`@property magnitude`	Convert your existing method to a property. Drop the parentheses everywhere it's called.
2	`__add__(self, other)`	Return a new Vector3D with components summed.
3	`__sub__(self, other)`	Return a new Vector3D with components subtracted.
4	`__mul__(self, scalar)`	Scalar multiply: each component × scalar. Returns a new Vector3D.
5	`__rmul__(self, scalar)`	Same result as __mul__ — makes 2 * v work.
6	`__eq__(self, other)`	True only if all three components match.
7	`__abs__(self)`	Return self.magnitude. abs(v) gives the norm.
8	`__len__(self)`	Return 3 — the number of components.

Keep dot(), normalize(), and __repr__ from yesterday. Save as the same file you used yesterday (rename to vector3d.py if you haven't).

Done when this script works

```python
from vector3d import Vector3D

a = Vector3D(1, 2, 3)
b = Vector3D(4, 5, 6)

print(a + b)
print(b - a)
print(a * 3)
print(3 * a)        # __rmul__ check

print(a == Vector3D(1, 2, 3))
print(a == b)

print(a.magnitude)  # no parens
print(abs(a))
print(len(a))
```

OUTPUT

```
Vector3D(5, 7, 9)
Vector3D(3, 3, 3)
Vector3D(3, 6, 9)
Vector3D(3, 6, 9)

True
False

3.7416573867739413
3.7416573867739413
3
```

Tips & gotchas

Don't write v.magnitude() anymore.

Once it's @property, the parens become a TypeError. Update normalize() and dot() too if they used the old style.

__rmul__ is a one-liner.

Just return self * scalar — don't rewrite the multiplication logic. Delegating to __mul__ keeps both in sync.

Every operator returns a new object.

v1 + v2 builds a third Vector3D. Never mutate self or other inside __add__, __sub__, etc.

__eq__ should never crash.

If other isn't a Vector3D (no .x attribute), Python expects you to return NotImplemented. For now just trust the inputs — proper handling is Thursday's exception lesson.

When the test script outputs everything cleanly, send the file back here.